

ShipOne: how it works

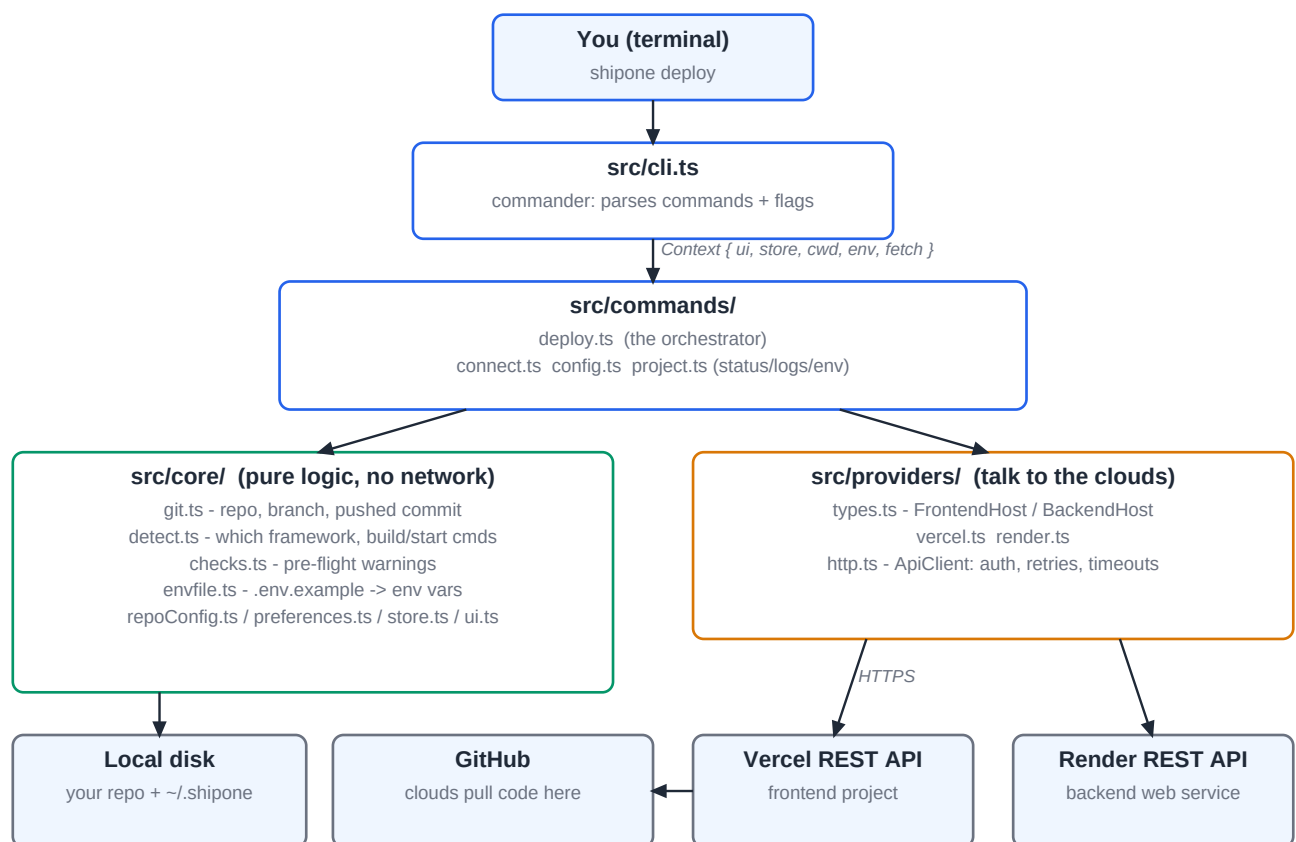
One command that deploys a full-stack app (frontend + backend) to your own Vercel and Render accounts, wired together.

1. The idea in one minute

You push your code to GitHub and run `shipone deploy`. ShipOne finds the frontend and backend folders, works out the framework of each, creates a Vercel project for the frontend and a Render web service for the backend, sets the environment variables so they can talk to each other (backend URL into the frontend, frontend URL into the backend's CORS setting), and deploys the exact commit on GitHub. ShipOne never builds anything itself: Vercel and Render build from GitHub. ShipOne's job is to **detect**, **configure** and **orchestrate**.

It is a local CLI written in TypeScript (Node 22+). There is no server and no database: tokens and state live in `~/.shipone/` on your machine.

2. Architecture



- **CLI layer** (`cli.ts`): turns `shipone deploy --dry-run` into a function call, and prints errors nicely.
- **Commands** (`commands/`): one file per user-facing feature. `deploy.ts` is the brain that calls everything else in order.
- **Core** (`core/`): pure logic that only reads files and git. Easy to unit-test because it never touches the network.
- **Providers** (`providers/`): one class per cloud, each behind an interface. Adding Netlify or Railway means one new class, no changes to deploy logic.

How the pieces communicate

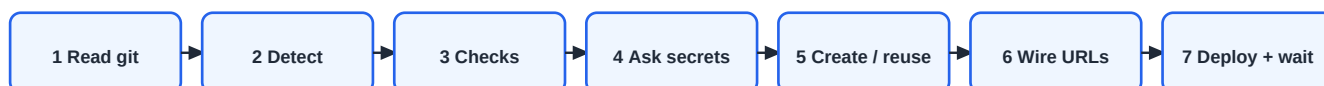
- **Dependency injection via a Context object.** Every command gets `{ ui, store, cwd, env, fetch }`. In production these are the real terminal, real files and real fetch; in tests they are a scripted UI, a temp folder and a fake HTTP server. Commands never read stdin or call the network directly.
- **Interfaces for clouds.** `FrontendHost` (`createProject`, `setEnv`, `deploy`, `getDeploy...`) and `BackendHost` (`createService`, `configure`, `deploy`, `logs...`). `deploy.ts` only knows these interfaces, not Vercel or Render.

- **ApiClient** (`http.ts`) is shared by both providers: bearer token auth, 30s timeout, retries on 429 (honouring `Retry-After`), 5xx and network blips, but a POST is never retried after a server error (it could create a duplicate).
- **Three kinds of storage:** `.shipone.yml` in the repo (settings you commit), `~/.shipone/credentials.json` (tokens, file mode 0600), and `~/.shipone/state.json` (ids and URLs of created services, so redeploys reuse them).

3. Which folder holds what

File / folder	What it does
<code>src/cli.ts</code>	Entry point (the shipone binary). Defines commands and flags with commander.
<code>src/commands/deploy.ts</code>	The orchestrator: builds a plan, shows it, asks for secrets, creates/reuses services, wires URLs, deploys and waits.
<code>src/commands/connect.ts</code>	Paste and verify a Vercel token / Render API key, pick a Render workspace.
<code>src/commands/config.ts</code>	Account defaults and per-repo overrides (which provider, Render region/plan).
<code>src/commands/project.ts</code>	<code>status</code> , <code>logs</code> , <code>env set</code> , <code>env ls</code> for services already created.
<code>src/core/git.ts</code>	Reads the GitHub owner/repo from the remote URL, current branch, HEAD, and whether it is pushed.
<code>src/core/detect.ts</code>	Stack detection: finds app folders, framework, package manager, build and start commands.
<code>src/core/checks.ts</code>	Pre-flight checks: hardcoded localhost URLs, backend ignoring <code>\$PORT</code> , nodemon in start, missing SPA rewrite.
<code>src/core/envfile.ts</code>	Parses <code>.env</code> / <code>.env.example</code> and decides which keys ShipOne fills itself and which it must ask for.
<code>src/core/repoConfig.ts</code>	Reads, validates and writes <code>.shipone.yml</code> .
<code>src/core/preferences.ts</code>	Chooses the provider: <code>.shipone.yml</code> , then per-repo override, then account default, else ask.
<code>src/core/store.ts</code>	Everything under <code>~/.shipone</code> (config, credentials, state).
<code>src/core/ui.ts</code>	Terminal UI (prompts, spinners) behind an interface so tests can script answers.
<code>src/providers/types.ts</code>	The FrontendHost / BackendHost interfaces and shared types (<code>DeployStatus</code> , <code>EnvVar...</code>).
<code>src/providers/vercel.ts</code>	Vercel REST API client (projects, env vars, deployments, domains).
<code>src/providers/render.ts</code>	Render REST API client (services, env vars one key at a time, deploys, logs).
<code>src/providers/http.ts</code>	Shared HTTP client with auth, retries, timeouts and readable errors.
<code>test/</code>	Unit, integration and end-to-end tests (section 7). <code>test/smoke/</code> builds real apps.
<code>docs/PLAN.md</code>	Design decisions, scope and roadmap.

4. What happens when you run shipone deploy



Steps 1-4 only read and ask. Nothing is created until every input is known (no half-created services).

- **1 Read git.** Find the GitHub repo and the commit that is actually on GitHub. If you have unpushed commits it stops or asks, because the clouds can only build what GitHub has.
- **2 Detect.** Find the frontend and backend folders and how to build/start them (section 5). `.shipone.yml` overrides detection.
- **3 Checks.** Warn about things that 'deploy fine but don't work', like `fetch('http://localhost:5000')` in the frontend.
- **4 Ask secrets.** Read `.env.example`. Keys ShipOne can fill (API URL, CORS, PORT) are automatic; real secrets are asked for once.

- **5 Create or reuse.** Reuse by saved id, else by name + linked repo, else create. Deploy-on-push is turned off on both sides.
- **6 Wire URLs.** Backend URL goes into VITE_API_URL (or the name your code already uses); frontend URL goes into CORS_ORIGIN / FRONTEND_URL.
- **7 Deploy and wait.** Both sides deploy the exact commit SHA; ShipOne polls status and prints the last log lines if a build fails.

The chicken-and-egg problem

The frontend needs the backend URL at build time, and the backend needs the frontend URL for CORS. Solution: create the Vercel project first (its <name>.vercel.app domain is known immediately), create the Render service with CORS already set (its URL is known on creation), then set the API URL on Vercel and deploy. If the frontend URL turns out different, update CORS and redeploy the backend once. Normally each side builds exactly once.

5. Stack detection (src/core/detect.ts)

Where it looks

- The repo root, every top-level folder, apps/*, packages/*, services/*, plus whatever the monorepo declares in pnpm-workspace.yaml or package.json "workspaces". Folders like node_modules, dist, build, tests, venv are skipped.
- Workspace globs are turned into regexes: * becomes [^/]* (one folder), ** becomes .* (any depth).
- For each folder: **frontend first**, then backend detectors in order **Node, Python, Go, Rust, Ruby, Docker**. The first one that matches wins.
- If there are several candidates, the obvious one is picked by folder name (client/web/frontend, server/api/backend), otherwise the user is asked.
- In a monorepo the root is ignored when a real app exists inside the workspaces (the root only holds tooling like turbo).

Frontends: one dependency decides

Dependency in package.json	Framework	API URL env var
next	Next.js	NEXT_PUBLIC_API_URL
@sveltejs/kit / astro	SvelteKit / Astro	PUBLIC_API_URL
nuxt	Nuxt	NUXT_PUBLIC_API_URL
@react-router/dev, @solidjs/start, vite	React Router 7, SolidStart, Vite	VITE_API_URL
react-scripts / @vue/cli-service	CRA / Vue CLI	REACT_APP_API_URL / VUE_APP_API_URL
gatsby, @angular/core, @remix-run/react	Gatsby, Angular, Remix	GATSBY_, NG_APP_, REMIX_PUBLIC_

Meta-frameworks are checked before vite because Astro, SvelteKit and others also depend on vite. A folder with a server dependency (express, fastify...) and vite but **no index.html** is a backend that uses vitest, not a frontend. If your code already reads a variable like VITE_BACKEND_URL, ShipOne reuses that name:

```
\b(VITE_[A-Z0-9_]*)(?:API|BACKEND|SERVER)[A-Z0-9_]*(?:URL|URI|ORIGIN|HOST|BASE)[A-Z0-9_]*\b
```

Backends: signal file, then build and start command

Runtime	Detected by	Build command	Start command
Node	package.json with express, fastify, koa, hapi, @nestjs/core, hono... in dependencies	npm: npm ci yarn: yarn install --frozen-lockfile pnpm: npx pnpm@X install --frozen-lockfile + build script if present	npm start / yarn start / npx pnpm@X start, else node <main>
Python	requirements.txt, pyproject.toml or Pipfile	pip install -r requirements.txt poetry / uv / pipenv variants	FastAPI: uvicorn mod:app --host 0.0.0.0 --port \$PORT Flask/Django: gunicorn -b 0.0.0.0:\$PORT ...

Runtime	Detected by	Build command	Start command
Go	go.mod	go build -o app <main pkg>	./app
Rust	Cargo.toml	cargo build --release	./target/release/<bin>
Ruby	Gemfile	bundle install	Rails: bundle exec puma -C config/puma.rb Sinatra: ruby app.rb -o 0.0.0.0 -p \$PORT
Docker	Dockerfile	(image builds itself)	(image CMD)

6. How the build command is decided (the tricky parts)

Node package manager

- Lockfile decides the tool: pnpm-lock.yaml → pnpm, yarn.lock → yarn, package-lock.json → npm (app folder first, then repo root).
- No global installs.** Render's /usr/bin is read-only, so npm i -g pnpm and corepack enable fail with EROFS. pnpm runs through npx instead (this was the real jobPilot failure).
- The pnpm version is pinned from the "packageManager" field, or from the lockfile format so old lockfiles still work:

```
packageManager field: ^(npm|pnpm|yarn)@(\d+(?:\.\d+){0,2}[\^+\s]*)
  "pnpm@9.15.4+sha512..." -> npx pnpm@9.15.4
lockfile format:      lockfileVersion:\s*['"]?(\\d+)
  5 -> pnpm@7, 6 -> pnpm@8, 9 -> pnpm@latest-10
```

- Yarn 2+ (lockfile starts with __metadata: or .yarnrc.yml exists) runs via npx -p @yarnpkg/cli-dist@4 yarn, since Render only has Yarn 1.
- A frozen install (npm ci, --frozen-lockfile) is used only when the lockfile really belongs to the app: its own, or the workspace root's when the app is a workspace member.
- Monorepo dependencies:** if the backend imports "@repo/shared": "workspace:*", the shared package must be built first: npx pnpm@X --filter "<app>..." run --if-present build (the ... means 'and everything it depends on', in order).

Python

- Tool: poetry.lock → Poetry (poetry install --no-root, start with poetry run), uv.lock → uv (uv sync --frozen, start with uv run --no-sync), requirements.txt → pip, Pipfile → pipenv, only pyproject.toml → uv pip install -r pyproject.toml.
- Requirements files are followed through -r other.txt includes. The app object is found with a regex, which also handles app/main.py and src/ layouts:

```
app object: ^(\w+)\s*(?:[:^=\n]+)?=\s*(?:\w+\.)?(?:FastAPI|Starlette|Litestar|Quart|Flask)\(
Flask factory: ^def create_app\(\s*-> gunicorn "app:create_app()"
Django:      DJANGO_SETTINGS_MODULE['"]\s*,\s*['"](\w+\.)['"] in manage.py -> <pkg>.wsgi
includes:    ^\s*(?:-r|--requirement)\s*=\s*(\S+)
```

- If FastAPI has no uvicorn (or Flask/Django no gunicorn), ShipOne adds it to the build, because the dev server only listens on localhost.

Go, Rust, Ruby

- Go:** framework from go.mod (gin-gonic/gin, labstack/echo, gofiber/fiber, go-chi/chi). The main package is the folder with ^package main\b: root, then cmd/server, cmd/api..., then cmd/. If an app/ folder exists the binary goes to .bin/app (otherwise Go writes it inside the folder).
- Rust:** binary name from [package] default-run, then [[bin]] name, then [package] name (read per TOML section, not the first name = in the file). Cargo workspaces use the shared ./target.
- Ruby:** gems matched with gem\s+['"]NAME['"]. Rails uses its config/puma.rb (rackup isn't bundled with Rails 7.1+); Sinatra is bound to 0.0.0.0:\$PORT.

Pre-flight checks (src/core/checks.ts)

- Hardcoded local URLs:
(https?|wss?|mongodb|postgres|mysql|redis)://(localhost|127.0.0.1|0.0.0.0), reported as file:line,

skipping config and test files.

- Backend ignores \$PORT: per-language patterns like `process.env.PORT`, `os.Getenv("PORT")`, `os.environ["PORT"]`. Skipped when the start command already passes \$PORT.
- Listening on localhost only, nodemon in start, and React Router without a Vercel rewrite (refreshing /about would 404).

7. Testing

Tests run with **Vitest** (`npm test`) and in GitHub Actions on Node 22 and 24. There are 130 tests in 5 files, plus a separate smoke suite.

Layer	File	What it proves
Unit	<code>test/core.test.ts</code> (39)	git URL parsing, <code>.shipone.yml</code> validation, provider preference order, 0600 credential file, <code>.env</code> planning, pre-flight checks.
Unit	<code>test/detect.test.ts</code> (50)	Real-world repo shapes per stack (pnpm/turbo monorepos, Yarn 4, FastAPI app/ layout, Django settings package, Go cmd/, Cargo workspace...) → the exact build/start command. Final loop: every popular stack must have a start command and no global install.
Provider	<code>test/providers.test.ts</code> (13)	The HTTP client (retries, timeouts, errors) and Vercel/Render clients, checked against request shapes from their official API specs.
Integration	<code>test/deploy.test.ts</code> (24)	Whole deploy flow against <code>fakeCloud.ts</code> , an in-memory fake of both APIs: first run, redeploy, lost state, deleted service, name clash, build failure, missing secret, dry run.
End-to-end	<code>test/cli.e2e.test.ts</code> (4)	Runs the real CLI as a subprocess over HTTP against the fake APIs.
Smoke (real)	<code>npm run smoke</code>	Builds and boots a real app for each stack (below).

How the test cases were designed

- **Start from real failures.** The jobPilot monorepo failed on Render (EROFS from `corepack enable`). That became a test: pnpm must never install globally.
- **Mirror real projects.** Each fixture copies what the framework's generator or a typical repo puts on disk (e.g. FastAPI's `app/main.py`, Django's `config/settings/`).
- **Assert the exact command**, not just 'something was detected', so a change in any command is caught.
- **Prove the tests catch bugs:** the new detection tests were run against the old code first, and 35 of 49 failed.
- **Isolation:** every test writes files into a fresh temp folder; git repos are created with a fake 'pushed' remote; the network is replaced by the fake cloud server.

Smoke test: does the command actually work?

A string match can't prove a build works, so `test/smoke/run.ts` goes further. For each of 20 fixtures it writes a small real project, generates lockfiles the way a developer would, runs detection, then runs the detected **build** and **start** commands in a Render-like environment: node, npm and Yarn 1 on PATH but **no pnpm**, Python inside a virtualenv, \$PORT set. It passes only if the server answers HTTP on that port.

Stacks covered by the smoke suite	Old detector	New detector
Express (npm), Hono (yarn), npm workspaces + Koa, Go chi cmd/api, Sinatra app.rb, uv at repo root	pass	pass
pnpm monorepo + TypeScript Fastify + workspace package (jobPilot shape), Fastify with vite devDeps	backend not found	pass
Yarn 4 Plug'n'Play	build fails (Yarn 1)	pass
FastAPI app/main.py, Flask create_app, Django settings/, Pipfile	no start command	pass
FastAPI src/ without uvicorn, uv in subfolder, Go with app/ folder, Sinatra config.ru, Rust workspace	server never starts	pass
pyproject.toml only, Poetry 2 project	build fails	pass
Total	6 / 20	20 / 20

```
npm test                # all unit/integration/e2e tests (fast, no network)
npm run typecheck
npm run smoke           # real builds (slow, needs node/python/go/ruby)
npm run smoke -- --all  # include Rust
npm run smoke -- fastapi go # filter by name
```

8. Design decisions worth mentioning

- **Nothing is created until every input is known**, so a missing secret can't leave half-created services behind.
- **Idempotent**: running deploy twice reuses services (saved id → name + repo → create).
- **Render env vars are set one key at a time**: the bulk endpoint replaces all vars and would wipe ones set in the dashboard.
- **Exact commit**: deploys the SHA that is on GitHub, not local HEAD. Pushing never deploys; only shipone deploy does.
- **Tokens, not OAuth**: OAuth needs a client secret, which can't live safely in a CLI. A pasted token works without a server.
- **Extensible**: new framework = one detector in detect.ts; new cloud = one class implementing FrontendHost or BackendHost.
- **Known gaps**: npm/yarn workspace packages that need building (only pnpm is handled), Rails asset precompile, and apps where frontend and backend share one package.json.